

Funzioni in JS memoria

Programmazione

Le funzioni nella memoria di JavaScript

In JavaScript, una funzione è un blocco di codice riutilizzabile che viene definito e successivamente eseguito attraverso una chiamata. Quando una funzione viene dichiarata, il motore JavaScript ne crea una rappresentazione in memoria, che può essere richiamata durante l'esecuzione del programma.

Ogni chiamata genera un nuovo contesto di esecuzione, nel quale vengono gestiti parametri, variabili locali e istruzioni. Le informazioni necessarie all'esecuzione vengono organizzate attraverso lo stack delle chiamate (Call Stack), mentre oggetti e altri dati possono essere allocati nell'area di memoria comunemente chiamata Heap.

Le variabili locali e il ciclo di vita della memoria

Quando una funzione viene eseguita, le variabili dichiarate al suo interno appartengono al relativo ambito di visibilità (Scope).

Queste variabili possono essere utilizzate durante l'esecuzione della funzione, ma normalmente non sono accessibili dall'esterno. Al termine della chiamata, il contesto di esecuzione viene rimosso dal Call Stack.

Tuttavia, questo non significa che tutti i dati creati vengano immediatamente eliminati: JavaScript utilizza un sistema automatico chiamato Garbage Collector, che recupera la memoria occupata dai dati non più raggiungibili dal programma.



Le funzioni come strumenti per mantenere dati in memoria

Una caratteristica importante di JavaScript è la possibilità di trattare le funzioni come veri e propri valori: possono essere assegnate a variabili, passate come parametri e restituite da altre funzioni.

Attraverso un meccanismo chiamato Closure, una funzione interna può mantenere accessibili le variabili dell'ambiente in cui è stata creata, anche dopo il completamento della funzione esterna.

Questa caratteristica permette di realizzare contatori, gestire stati interni e conservare informazioni tra chiamate successive, senza ricorrere necessariamente a variabili globali. I dati così mantenuti rimangono nella memoria del programma finché risultano raggiungibili.



Esercizio 1 – Variabili locali

Realizzare un programma JavaScript che definisca una funzione chiamata calcolaTotale(), capace di ricevere due numeri come parametri, sommarli e restituire il risultato. Richiamare la funzione almeno tre volte utilizzando numeri differenti e stampare i risultati nella console.

Verificare successivamente se è possibile accedere dall'esterno a una variabile dichiarata con let all'interno della funzione, spiegando il motivo del comportamento osservato.



```
1.
2. <!DOCTYPE html>
3. <html lang="it">
4. <head>
5.   <meta charset="UTF-8">
6.   <title>Esercizio 1 - Variabili locali</title>
7.   <style>
8.     body {
9.       font-family: Arial;
10.      padding: 30px;
11.      background-color: #f2f2f2;
12.    }
13.
14.    button {
15.      padding: 10px;
16.      cursor: pointer;
17.    }
18.  </style>
19. </head>
20. <body>
21.
22.  <h2>Esercizio 1 - Variabili locali</h2>
23.
24.  <button onclick="esegui()">Calcola</button>
25.
26.  <p id="risultato"></p>
27.
28.  <script>
29.    function calcolaTotale(a, b) {
30.      let somma = a + b;
31.      return somma;
32.    }
33.
34.    function esegui() {
35.      let risultato1 = calcolaTotale(10, 20);
36.      let risultato2 = calcolaTotale(5, 15);
37.      let risultato3 = calcolaTotale(30, 40);
38.
39.      document.getElementById("risultato").innerHTML =
40.        risultato1 + "<br>" +
41.        risultato2 + "<br>" +
42.        risultato3;
43.
44.      // La variabile somma esiste solo
45.      // all'interno di calcolaTotale()
46.
47.      // console.log(somma); // ReferenceError
48.    }
49.  </script>
50.
51. </body>
52. </html>
53.
```

Esercizio 2 – Memorizzare un valore tra chiamate

Realizzare un programma JavaScript che permetta di mantenere in memoria il numero di volte in cui una funzione viene richiamata.

Utilizzare una variabile esterna e una funzione chiamata incrementaContatore(), che aumenti il valore del contatore di un'unità a ogni esecuzione. Stampare il valore aggiornato dopo ogni chiamata e verificare che il dato non venga reinizializzato.



```
1.
2. <!DOCTYPE html>
3. <html lang="it">
4. <head>
5.     <meta charset="UTF-8">
6.     <title>Esercizio 2 - Contatore</title>
7.     <style>
8.         body {
9.             font-family: Arial;
10.            padding: 30px;
11.            background-color: #f2f2f2;
12.        }
13.
14.        button {
15.            padding: 10px;
16.            cursor: pointer;
17.        }
18.    </style>
19. </head>
20. <body>
21.
22.    <h2>Esercizio 2 - Contatore</h2>
23.
24.    <button onclick="incrementaContatore()">
25.        Incrementa
26.    </button>
27.
28.    <p id="risultato">Contatore: 0</p>
29.
30.    <script>
31.        // Variabile esterna alla funzione
32.        let contatore = 0;
33.
34.        function incrementaContatore() {
35.
36.            // Modifica della variabile esterna
37.            contatore++;
38.
39.            document.getElementById("risultato").innerHTML =
40.                "Contatore: " + contatore;
41.        }
42.    </script>
43.
44. </body>
45. </html>
46.
```



Esercizio 3 – Closure e memoria privata

Realizzare una funzione chiamata creaContatore() che inizializzi una variabile numerica a zero e restituisca una seconda funzione.

Ogni volta che la funzione restituita viene eseguita, deve incrementare il numero memorizzato e restituire il nuovo valore. Creare due contatori indipendenti e verificare che ciascuno conservi il proprio stato senza interferire con l'altro.

L'obiettivo è dimostrare come una Closure permetta di conservare informazioni tra chiamate successive, mantenendo separati gli stati delle diverse istanze.



```
1.
2.<!DOCTYPE html>
3.<html lang="it">
4.<head>
5.    <meta charset="UTF-8">
6.    <title>Esercizio 3 - Closure</title>
7.    <style>
8.        body {
9.            font-family: Arial;
10.           padding: 30px;
11.           background-color: #f2f2f2;
12.       }
13.
14.       button {
15.           padding: 10px;
16.           margin: 5px;
17.           cursor: pointer;
18.       }
19.   </style>
20.</head>
21.<body>
22.
23.   <h2>Esercizio 3 - Closure</h2>
24.
25.   <button onclick="esercizio3A()">
26.       Incrementa A
27.   </button>
28.
29.   <button onclick="esercizio3B()">
30.       Incrementa B
31.   </button>
32.
33.   <p id="risultatoA">Contatore A: 0</p>
34.   <p id="risultatoB">Contatore B: 0</p>
35.
36.   <script>
37.       function creaContatore() {
38.
39.           // Variabile locale della funzione esterna
40.           let numero = 0;
41.
42.           // Restituzione di una funzione interna
43.           return function() {
44.               numero++;
45.               return numero;
46.           };
47.       }
48.
49.       // Creazione di due contatori indipendenti
50.       let contatoreA = creaContatore();
51.       let contatoreB = creaContatore();
52.
53.       function esercizio3A() {
54.           let valore = contatoreA();
55.
56.           document.getElementById("risultatoA").innerHTML =
57.               "Contatore A: " + valore;
58.       }
59.
60.       function esercizio3B() {
61.           let valore = contatoreB();
62.
63.           document.getElementById("risultatoB").innerHTML =
64.               "Contatore B: " + valore;
65.       }
66.   </script>
67.
68.</body>
69.</html>
70.
```


Buon davante a tutti